Date
Page 1

SQL (PRACTICAL FILE)

(Here, "student" is the table name)

1. (i) To show information of all students of history department.
   → select * from student where department = history";
   (ii) To list the names of female students who are in Hindi Department.
   → select Name from student where Gender = "f" and
   Department = "Hindi";
   (iii) To display the names of all students where their date of admission in ascending order.
   → select Name, Date of adm from student order by Date of Adm as c;
   (iv) To count the number of student who were ~~born~~ admitted in 1998.
   → select count (*) as Number_of_students_admitted_in_1998 from student where Date of adm like "1998-%";
   (v) To select the names of all departments.
   → select distinct departments from student;

Rajveer Singh

P. T. O.

1) Student

No	Name	Age	Department	Date of adm	Fee	Gender
1	Pankaj	24	Computer	1997-01-10	120	m
2	Shalini	21	History	1998-03-24	200	f
3	Sanjay	22	Hindi	1996-12-12	300	m
4	Sudha	25	History	1999-07-01	400	f
5	Rakesh	22	Hindi	1997-09-05	250	m
6	Shakeel	30	History	1998-06-27	300	m
7	Surya	34	Computer	1997-02-25	210	m
8	Shikha	23	Hindi	1997-07-31	200	f

(i) Display the names of those whose Date of admission is the year 1998.

→ select Name from student where Date of Adm like "1998-%";

(ii) Display the number of departments which start with letter 'H'.

→ select count(*) as No_of_Dep_from H from student where Department like 'H%';

(iii) Display the number of those departments which end with 'y'.

→ select count(*) from student where Department like '%y';

(iv) Display the names of those students where 'Day' part of Date of Adm starts with number '1'.

→ select Name, Date of Adm from Student where Date of Adm like "%-%-1%";

Name	Date of Adm
Pankaj	1997-01-10
Sanjay	1996-12-12

(v) Display the names, along with their Date of Adm of those students where their 'Day' part of Date of Adm has the 2nd digit as '1'.

→ select Name, Date of Adm from student where Date of Adm like "%-%-%1";

Date is in the form of: yyyy-mm-dd.

Name	Date of Adm
Sudha	1999-07-01
Shikha	1997-07-31

(vi) Select Name from student where Department like "%w";
→ Empty set

(vii) Display all the departments as given in the table.
→ select Departments from student;

(viii) Display all the Ages as the same order in the table.
→ select Age from student;

(ix) Display the different ages in the same order in the table.
→ select distinct Age from student;

(x) Display all the Ages in the table, but in order wise.
→ select ~~Department~~ Age from student order by ~~Department~~ Age;

(xi) Display the different ages in the table in serial order.
→ select distinct ~~Department~~ Age from student order by Age;
or
→ select Age from student group by Age.

Rajveer Singh

(viii)	(ix)	(x)	(xi)
24	24	21	21
21	21	22	22
22	22	22	23
25	25	23	24
22	30	24	25
30	34	25	30
34	33	30	34
33		34	

2) (i) Create a database /or use it if it is already made.
Create the given table/or open the table if already exist.
→ enter password:
show databases;
use <database_name>;
Show tables;
select * from <tablename>.

create database <database_name>;
Use <database_name>;
create table <table_name> (fieldname datatype, " ", " ");
insert into <table_name> values (for number, "for string");

(ii) create table CARDEN (Ccode int, CarName varchar(18),
make varchar(18), Colour varchar(18)
Capacity int, charges float)
insert into CARDEN values (501, "A-Star", "Suzuki", "RED", 3, 14.5);
insert into CARDEN values (510, "CClass", "mercede", "Blue", 4, 35.9);

(iii) To display all information about the Suzuki Cars.
→ select * from CARDEN where make = "Suzuki";
(iii) To display the Ccode, CarName, and colour in descending order of their capacity.
→ select Ccode, CarName, Colour from CARDEN order by capacity desc;
(iv) To display the names of all the red coloured cars that can accomodate 4 people.
→ select CarName from CARDEN where Capacity = "4"
And colour = "RED";

Rajveer Singh

(v) To display the total number of models (make).
→ Select count(distinct(make)) as Total_No_of_Models from CARDEN; ~~group by make~~

(vi) To display the information of cars where the name starts with letter 'I' is ascending order of CarName.
→ select * from CARDEN where CarName like "I%" order by CarName;

(vii) To display the names of these cars where the capacity is more than 4 and Ccode is more than 505.
→ Select CarName from CARDEN where Capacity > 4 and Ccode > 505;

(viii) To display the no. of cars of each make.
→ Select make, count(make) as No_of_cars from CARDEN group by Make;

3) (i) Display the TName, TCode, IName from the tables.
→ Select T.TName, T.TCode, I.IName from traders T, items I where T.TCode = I.TCode;

(ii) Display the IName, TCode, TName, where quantity is less than 100.
→ Select I.IName, I.TCode, T.TName from traders t, items i where T.TCode = I.TCode and I.Qty < 100;

(iii) Display the TName and the ~~total~~ number of items that each trader ~~bought, in ascending order~~.
→ ~~Select T.TName, count(IName) as Number_of_items from traders T, items I where T.TCode = I.TCode order by asc;~~
→ ~~Select T.TName, sum(QTY) as No_of_items from traders t, items I where T.TCode = I.TCode Order by sum(QTY) as c, group by T.TName;~~
→ Select T.TNAME, Sum(QTY) as Number_of_items from trader T, items I where T.TCode = I.TCode group by T.TNAME;

(iv) Display the total nombo of companies each trader has bought.
→ Select T.TNAME, ~~I.COMP~~ count(COMPANY) from traders t, items I where T.TCode = I.TCode group by T.TNAME;

4) (i) To display the Name and Price of the accessories in ascending order of its price;
→ Select IName, Price from accessories order by price asc;
(ii) To display the ID and SName of all shops located in Nehru Place.
→ Select ID, SName from Shops where Area = "Nehru Place";
(iii) To display the max and min price of each name of accessories.
→ Select IName, max(Price) as maximum-Price, min(Price) as minimum Price from accessories group by Iname;
(iv) To display the Name, Price of all the accessories and its respective SName where they are available.
→ select I.IName, I.Price, S.SName from accessories I, shop S where I.SNo = S.ID;
(v) Display the total number of items that each shop sells along with the shop names.
→ select S.SName, count(A.IName) from shop S, Accessories A, where S.ID = A.SNo group by A.IName;

5) (i) To display the frequency of employees department wise
→ select Depid, count(Name) from employee group by Depid;
(ii) To list the names of those employees only where name starts with 'H'.
→ select Name from employee where Name like 'H%';
(iii) To add a new column in salary table. The column name is Total Sal.
→ Alter table employee add Total_Sal float;
(iv) To show the corresponding values in Total_sal column.
→ Update employee set Total Sal = Basic + DA + HRA + Bonus;
(v) To add the percentage of bonus in the Total_Salary.
→ alter table salary add percentage float;
→ update salary set percentage = (Bonus * 100)/ Total_Sal;

Rajveer Singh

SCHOOL

QP. (IMPORTANT)(ONLY)

~~(i) To display the names of all th~~

6) (ii) To display the sum of prize money for the activities played in each of the stadium separately.

→ Select Stadium, Sum(PrizeMoney) from ACTIVITY group by ~~ActivityName~~ Stadium;

(iii) To display the contents of all activity ~~names~~ for which schedule date is earlier than 01-01-2004 in ascending order of Participants Num.

→ select * from ACTIVITY where ScheduleDate < 20040101 order by Participants Num asc;

(v) To display Stadium wise ActivityName.

→ select Stadium, ActivityName from ACTIVITY order by Stadium;

7) (i) To give a discount of 10%. in the car charges for existing customers (who are in the customer table).

~~→ altertable CUSTOMER add Discount float;~~

~~→ update CUSTOMER set Discount~~

→ Select C.CustName, T.Charges-(Charge/10) as Discount from Customer C, Car T where C.Ccode = T.Code;

(ii) To display the Name and Make of cars whose charges is in the range 2000 to 3000 (both inclusive).

→ Selec- Cname, Make from CAR where Charges <=3000 and Charges >= 2000;

Rajveer Singh

ALTER AND UPDATE AND MODIFY..

1. To add primary key to a relation.
   → alter table <table name> add primary key <field_name>;

2. To add foreign key:
   → alter table <table2_name> add foreign key <attribute_name> references <table1_name> (<table1 primary key>);

3. To add constraint to an existing attribute:
   → alter table <table name> add <constraint_name> (<attribute_name>);

4. Add an attribute to the existing table:
   → alter table <table_name> add <attribute name> <datatype>;

5. Modify datatype of an attribute:
   → alter table <table name> modify <attribute name> <datatype>;

6. Modify constraint of an attribute:
   → alter table <table name> modify <attribute name> <datatype> <constraint>;

7. Remove an attribute:
   → alter table <table name> drop <attribute name>.

8. Remove primary key from the table:
   → alter table <table_name> drop primary key;

9. Remove a constraint from the table:
   → alter table <table_name> drop <constraint> <attribute name>;

10. Set the value of a new field name added.
    → alter table <table_name> add <attribute name> <datatype>;
    → Update ~~table~~ <table_name> set <attribute name> =

DIFFERENCE BETWEEN DISTINCT, ORDER BY, GROUP BY.

Suppose in table it displays:

So, just selecting will show all the ages in the same order as given in the table.

Distinct will show all the different ages in the same order as in the table (i.e, those ages already displayed won't be displayed again)

order by will show all the ages in alphabetical/serial order asc/desc; group by will show all the distinct ages in serial order.

i.e;

Age
24
21
22
25
24
22
30
34
23

1. Select Age	2. Select Distinct Age	3. Select Age, Order by Age	4. Select Distinct Age, order by age
24	24	21	21
21	21	22	22
22	22	22	23
25	25	23	24
24	30	24	25
22	34	24	30
30	23	25	34
34		30	
23		34	

Distinct ↗

order ↗

distinct and order ↗

Pg 519
Event type

COMMANDS AND BASIC THEORY.

Commands
select *
Count *
as name
group by
order by
distinct
alter
update
set
drop
like
%
create
show
use
describe
insert-values.
LIMIT n

Constraints:
Default
not null
unique
primary key
foreign key
check
index
if null
candidate key

aggregate func:
max
min
sum
avg
count

Datatype:
char(4)
varchar(4)
int
float
date (YYYY-MM-DD)
time (HH-MM-SS)

joins:
cartesian product of two table
equi-join
natural-join
cardinality and degree and modality
dbms and rdbms

normal:
aliasing
in (or)
between (and)
meaning of null, is null, is not-null
update command
having clause.

Operators.
mathematical
relational
logical

While Describing:

Field	Type	Null	Key	Default	Extra.
		YES		NULL	

(Other different keys.

Equi-Join: The join in which columns are connected compared for equality is called equi-join.

Natural-join: The join in which only one of the identical columns (coming from joined tables) exists, is called Natural Join.

Having Clause: places conditions on groups
can include aggregate functions unlike Where.

FILE HANDLING : BASIC THEORY.

- To create, update, read and delete files through python.

TEXT FILES	BINARY FILES.
i) Stores information is ASCII format (Unicode)	i) Stores information in the same format as stored in the memory.
ii) Each line of text is ended with a special character known as EOL (End of line).	ii) No delimer occurs in binary files.
iii) Some internal translations take place when this EOL character is read or written.	iii) No translation occurs in binary files.
iv) Are slower than binary files.	iv) Binary files are faster and easier for a program to read and write the text files.

#Opening a file:

Syntax : file_object = open ("<filename.txt>", "<file_mode>")

file_object → It Acts as a link to access the contents of the opened file.

open() → It is a pre-defined function to open the named file and attach it with the file object. It takes 2 parameters the second parameter being optional for reading operation.

"<filename.txt>" → Name of the file to be opened. The location of the file is the current working directory.

"<file_mode>" - This means the operation to be performed on the opened file. The 3 basic operations are read, write and append.

If the file_mode is not mentionned in the open() function, the default file_mode is taken to be read mode.

file object → is it a stream of bytes which associates itself with the file opened and every function on the file is associated through this file object.

File Paths: → Absolute path : full path name
→ Relative path : only name of file as it is present in the current working directory.

Syntax for absolute path:

file_obj = open("<C:\\new_folder\\file_name.txt>", "file_mode")

The double slash ("\\") in the path name is needed because a single slash ("\") in a string can be interpreted as an escape sequence. There is an alternate way to write the path name with single slash ("/")

Synatx: file_obj = open (r"C:\new_folder\file_name.txt", "file_mode")

Synatax for Relative : file_obj = open ("filename.txt", "file_mode")

~~Text files~~

~~'r'~~
~~'w'~~
~~'a'~~

~~Binary file~~

~~'rb'~~
~~'wb'~~
~~'ab'~~

Tell()

It returns the current position (in bytes) of cursor in file
file_obj.tell()

Seek()

Changes the cursor position by ~~type~~ bytes as specified
file_obj.seek()

Text file.	Binary file.	Description
'r'	'rb'	• Opens the file in 'read' mode • Shows an error if file does not exist. • The file pointer is placed at the beginning (the first byte) of the file.
'w'	'wb'	
'a'	'ab'	
'r+'	'rb+'	
'w+'	'wb+'	
'a+'	'ab+'	

- for r/rb/r+/rb+ : throws error if file does not exist, file pointer at start.
- for w/wb/w+/wb+ : new file made if does not exist from before
  file pointer is placed in the beggining (the first byte) of the file
  if the file already exist, the existing data gets over written with new data.
- for a/ab/a+/ab+ : new file is made if file does not exist from before, and file pointer is placed in the beggining (first byte) of the file.
  if the file exists, additional data will be added/appended to the file after the existing data.

<<PDF>>

Closing a file:

Closing A file is a good programming practise as:

i) If the file is not closed and the program ends unexpectedly, the data might not be written to the file.

ii) ~~After the close~~

* In CSV, without newline="", there will be an extra [ ] between every data line.

Reading a Binary file:

```
import pickle
f3=open("Book.dat","rb")
try:
  while True:
      s=pickle.load(f3)
      print(s)
  except EOFError:
      print(" ")
f3.close()
```

Reading a CSV file:

```
import csv
f=open("Stu.csv","r")
csvr=csv.reader(f)
for i in csvr:
    print(i)
```

For text files:

1) file_object = open("filename.txt", "file_mode")
2) f1.read(n) → for whole file, n → how much from starting of file.
3) f1.readline(n) → displays first line,
   n → how much from starting but only inside first line.
3) f1.readlines(n) → each line as an element in a list.
4) for i in f1 → to print no. of lines from a file, print lines one by one.
5) f1.write(<string>)
6) f1.writelines(<list>) → each element of the list is a string to be written in the file.
7) s=f1.read()
   for i in s.split():
       print(i) → all words in each line.

* No need for "f.close" with files opened in "with open" form.

For binary files:

1. pickle.dump(l, f1) → to write 'l' in 'f1'.
2. j = pickle.load(f1) → to read 'f1' file.
3. os.remove(f1) → to delete a file

for csv files:

1. csv.reader() — To read the records
2. csv.writer() — To write records.

File Text Examples: suppose: abcdefg
hi jk lmn o
pqr stuv

Then; 1. f = open("text.txt","r")
for i in f:
print(i) ⟶ (same info but space in between)

output: abcdefg

hi jk lmn o

pqr stuv

f = open("text.txt","r")
s = f.read()
for i in s:
print(i) → all the letters of the file, one letter in one line.

2. f = open("text.txt", "r")
s = f.read()
print(s) → (exactly same)

Output: abcdefg
hi jK lmn o
pqr stuv

3. f = open("text.txt", "r")
s = f.readline() → (only first line)
print(s) output: abcdefg

4. f = open("text.txt", "r")
s = f.readline(2) → (only first 2 elements of first line)
print(s) output: ab

5. f = open("test.txt", "r")
s = f.readline(12) → (12 digits but only in first line).
print(s) output: abcdefg

6. s = f.readlines() → whole text file in a list with each line as element.
print(s)
output: ~~['abcdefg\n' 'hi jK lmn~~
['abcdefg\n', 'hi jK lmn o\n', 'pqr stuv']

7. s = f.readlines(2) → only the first line
print(s)
output: ['abcdefg\n']

8. s = f.read(5) → only first 5 letters
print(s) → output: abcde

SAMPLE Qs:

1. To read information from a text file:

```
f = open("text.txt","r")
for i in f:
    print(i)
```

or

```
s = f.read()
print(s)
```

2. To read first 5 letters of the text file:

```
f = open("text.txt","r")
s = f.read(5)
print(s)
```

3. To read first 4 lines of a text file:

```
f1 = open("text.txt","r")
c=0
for i in f1:
    if c==4:
        break
    print(i)
    c=c+1
```

or,

```
for i in f1:
    if c!=4:
        print(i)
        c=c+1
```

4. To count Number of lines in a text file:

```
f1 = open("text.txt","r")
c=0
for i in f1:
    c=c+1
print("Number of lines:", c)
```

5. To print only those lines which start with vowels:

```
f1 = open("text.txt","r")
for i in f1:
    if i[0] in 'AEIOUaeiou':
        print(i)
```

6. To print no. of vowels in the whole file:

```
f1 = open("text.txt", "r")
C=0
for i in f1:
    for w in i:
        if w in 'AEIOUaeiou':
            c=c+1
print("No. of vowels:", c)
```

or

```
c=0
s=f.read()
for i in s:
    if i in "AEIOUaeiou":
        c=c+1
print(c)
```

7. To display Number of 'A' or 'a' occuring in a text file:

```
f1 = open("text.txt", "r")
c=0  s=f.read()
d=0  for i in s:
        if i=="A":
            c=c+1
        elif i=='a':
            d=d+1
print("No of A:", c)
print("No of a:", d)
```

7.5. To display only those words which start with vowels.

```
f1=open("poem.txt","r")
s=f1.read()
w=s.split()
for i in w:
    if i[0] in 'AEIOUaeiou':
        print(i)
```

or

```
f1=open("poem.txt","r")
for i in f1:
    w=i.split()
    for j in w:
        if j[0] in 'AEIOUaeiou':
            print(j)
```

8. To write name, ~~rollno, marks in a file as a list~~ in a text file.

```
f= open("text.txt", "w")
~~for i~~
name = input("Enter Name:")
f.write(name)
f.close()
```

9. Write a python program to copy all information from one file to another file "copy.txt":

```
f1 = open("text.txt","r")
f2 = open("copy.txt","w")
S = f.read()
f2.write(S)
f1.close()
f2.close()
```

or

```
for i in f1:
    f2.write(i)
```

10. To display only the "Even number lines" of the file.

```
f1 = open("text.txt","r")
c=1
for i in f1:
    if c%2==0:
        print(i)
    c=c+1
```

or

```
c=0
for i in f1:
    if c%2!=0:
        print(i)
    c=c+1.
```

Q) Write a method/function DISPLAYWORDS() in python to readlines from a textfile STORY.Txt, and display those words which are less than 4 characters.

→
```
def DISPLAYWORDS():
    f1 = open("STORY.txt","r")
    for i in f1:
        w = i.split()
        for j in w:
            if len(j) < 4:
                print(w)
    f1.close()
DISPLAYWORDS()
```

Q) WAP to read characters from the keyboard one by one. All lowercase characters should be stored in a file LOWER, all uppercase characters in UPPER and all other characters inside the file OTHERS.

→
```
f1 = open("LOWER.txt","w")
f2 = open("UPPER.txt","w")
f3 = open("OTHERS.txt","w")
L = input("Enter a list")
for i in L:
    if i.isupper():
        f1.write(i)
    elif i.islower():
        f2.write(i)
    else:
        f3.write(i)
f1.close()
f2.close()
f3.close()
```

Q) To count the occurance of 'the' in a text file, irrespective of whether it is an individual word, or between a word.

```
f = open ("poem.txt","r")
s = f.read()
r = str(s)
print(r.count("THE"))
print(r)
```

Q) (Using open()):
A text file contains alphanumeric text (poem.txt). WAP that reads this text file and prints only those words that contain digits/numbers from the file.

```
f1 = open("poem.txt", "r")
for i in f1:                          #every line of file
    w = i.split()                     # w is all the words in that line
    for j in w:                       # taking 1 word at a time from w
        for k in j:                   # taking 1 digit from 1 word
            if k.isdigit and len(j)>1:
                print(j)
                break
```

or

```
f1 = open("poem.txt", "r")
for
s = f1.read()                         # s stores whole file.
w = s.split()                         # every word in that file.
for j in w:                           # 1 word at a time from w
    for k in j:                       # 1 digit from 1 word.
        if k.isdigit and len(j)>1:
            print(j)
            break
```

CONFUSION:

Suppose, text file: fdsf wOrd n1ght from
you m5y get n7 84na
L41i 9

&, 1) s = f.read()
w = s.split()
print(w) → ['fdsf', 'wOrd', 'n1ght', 'you', 'm5y' ... 'L41i', '9']

2) for i in f1:
w = i.split()
print(w) → ['fdsf', "word", 'n1ght', 'from']
['you', 'm5y', 'get', 'n7', '84na']
['L41i', '9']

3) s = f.read()
w = s.split()
for i in w:
print(i) →
fdsf
wOrd
n1ght
from
you
m5y
get
n7
84na
L41i
9

or,
for i in f1: →
w = i.split()
for j in w:
print(j)

5) s = f.read() → fdsfwOrdn... L41i9
for i in s:
print(i)

or →
for i in f1:
for j in i:
print(j)

use
(with open)

Q) Write a program to count total no. of 'the' word present in a text file.

→
```
with open ("poem.txt","r") as f1:
    c=0
    for i in f1:
        for w in i.split():
            if w.lower() == "the":
                c=c+1
    print("Total no. of the", c)
```

Q) Write a program that copies a text file "source.txt" to "target.txt" leaving the line starting with 'T'.

→
```
with open ("source.txt","r") as f1:
    with open ("target.txt","w") as f2:
        for i in f1:
            if i[0] not in 'Tt':
                f2.write(i)
```

Suppose in file "Test.txt", information is:

hello there, how are you?

```
fh = open ("Test.txt","r")
fh.seek (6)          # takes the file pointer at 6 byte.
x = fh.tell()        # 'tell' function tells where the file pointer is.
print(x)             that byte value is stored in x.
print (fh.read())
```

output : 6
hello there, how are you ?

BINARY.

Q) Write a function Createfile() to input n number of records in a binary file "Book.dat" with a structure: [BookNo, BookName, Author, Price], where n is given by the user.

→
```
import pickle
def Createfile(n):
    f1=open("Book.dat","wb")
    L=[]
    for i in range(n):
        BookNo = int(input("Enter BookNo :"))
        BookName = input("Enter Book Name :")
        Author = input("Enter Author Name :")
        Price = int(input("Enter price of book :"))
        L = [BookNo, BookName, Author, Price]
        pickle.dump (L, f1)
        L=[]
    f1.close()
n = int(input("Enter no. of records to be added :"))
Createfile(n)
```

Q) Write a function counting() that accepts an argument price. The function should print the total number of books that costs more than the value of the argument passed. Use the file book.dat created in the previous program.

→
```
import pickle
def Counting(price):
    f2 = open("Book.dat", "rb")
    c = 0
    try:
        while True:
            t = pickle.load(f2)
            if t[3] > price:
                c = c+1
                print(t)
    except EOFError:
        print("Total No. of books costlier than price:", c)
        f2.close()
price = int(input("Enter the price to be compared:"))
Counting(price)
```

Q) Write a function named Update_book() that will update the file "book.dat" created in the previous program. The function will change cost only. The new cost of the book will be given by the user.

→
```
import pickle
def Update_book(n,c):
    f1=open("Book.dat","rb")
    f2=open("Book.dat",
    L=[]
    try:
        while True:
            t=pickle.load(f1)
            if t[0]!=n:
                L.append(t)
            elif t[0]==n:
                t[3]=c
                L.append(t)
    except EOFError:
        f1.close()
        f2=open("Book.dat","wb")
        for i in L:
            pickle.dump(i,f2)
        f2.close()
n=int(input("Enter Book No:"))
c=int(input("Enter new cost:"))
Update_book(n,c)
```

Q) Write a function filter_book() that will copy the information about the book by the given author from "book.dat" to another file "Book_new.dat". After copying delete Book.dat. The name of the author will be passed as an argument to the function.

→
```
import pickle
import os
def EnterA(A):
    f1 = open("Book.dat","rb")
    f2 = open("Book_new.dat","wb")
    try:
        while True:
            t = pickle.load(f1)
            if t[2] == A:
                pickle.dump(t, f2)
            else:
                print("No Author found with the given name")
    except EOFError:
        f1.close()
        f2.close()
        os.remove("Book.dat")
A = input("Enter Name of Author to be found:")
EnterA(A)
```

Q) WAP to enter the following records in a binary file :
Item No integer
Item_Name string
Qty integer
Price float
No. of records to be entered by the user. Read the file to Display the records in the following format:
Item No : Item Name : Quantity : Price per item : Amount
(Amount to be calculated as Price * Qty)

→
```
import pickle
def EnterR(R):
    f1=open("Source.dat","wb")
    L=[]
    for i in range(R):
        ItemNo = int(input("Enter Item No:"))
        Item_Name = str(input("Enter Item Name:"))
        Qty = int(input("Enter Quantity:"))
        Price = float(input("Enter price:"))
        Amount = (Price * Qty)
        L=[ItemNo, ItemName, Qty, Price, Amount]
        pickle.dump(L,f1)
    f1.close()
R = int(input("Enter Number of records:"))
EnterR(R)
f2 = open("Source.dat","rb")
try:
    while True:
        t=pickle.load(f2)
        print(t)
except EOFError:
    f2.close()
```

CSV

Q) Create a file (CSV) with n number of records and structure of each record is in the format [id, destination, days, fare]. N is provided by the user. Write another function that will read and print the records in the file.

→
```
import csv
def WriteRecords(n):
    f1 = open("travel.csv","w", newline="")
    csvw = csv.writer(f1)
    L=[]
    for i in range(n):
        idd = int(input("Enter id number:"))
        dest = input("Enter destination:")
        days = int(input("Enter days:"))
        fare = int(input("Enter fare:"))
        L = [idd, dest, days, fare]
        csvw.writerow(L)
        L=[]
    f1.close()
n = int(input("Enter No of Records:"))
WriteRecords(n)
f2 = open("travel.csv","r")
csvr = csv.reader(f2)
for i in csvr:
    print(i)
f2.close()
```

Q) Write a python function that reads the before created csv file and print the records where the destination takes less than 15 days. The function accepts the name of the file as the argument.

→
```
import csv
def Check(n):
    f1 = open("n", "r")
    csvr = csv.reader(f1)
    for i in csvr:
        if i[2] < 15:
            print(i)
    f1.close()
n = input("Enter file name:")
Check(n)
```

Q) WAP a csv file with same ~~data~~ structure as given in the previous q. and having tab as delimer. Also, read and print the records in the file.

→
```
import csv
def Createfile(n):
    f1=open("travel.csv","w",newline="")
    csvw = csv.writer(f, delimeter = "\t")
    for i in range(n):
        idd =
        dest =
        days =
        fare =
        L = [idd, dest, days, fare]
        csvw.writerow(L)
        L=[]
    f1.close()
n= int(input("Enter no. of records"))
Createfile(n)
def Reader():
    f2= open("travel.csv","r")
    csvr = csv.reader(f2, delimeter = "\t")
    for i in csvr:
        print(i)
    f2.close()
Reader()
```

Q) A CSV file by "Records.csv" has the structure [Player_ID, Name, Score].
a) Write a user defined function Createfile() to input data for a record and add to "Records.csv".
b) Write a function Search() to display the Names and Score of the players scoring the highest and the lowest.

→
```
import csv
def Createfile(n):
    f1=open("Records.csv","w",newline="")
    csvw=csv.writer(f1)
    L=[]
    for i in range(n):
        Player_ID = int(input("Enter player ID:"))
        Name = input("Enter Name:")
        Score = int(input("Enter Score:"))
        L=[Player_ID,Name,Score]
        csvw.writerow(L)
        L=[]
    f1.close()
n=int(input("Enter Number of records:"))
Createfile(n)
def Search():
    f2=open("Records.csv","r")
    csvr=csv.reader(f2)
t=[]
    for i in csvr:
        t.append(i[2])
    r=max(t)
    d=min(t)
    f2.close()
```

```
    :
    f3 = open("Records.csv","r")
    csve = csv.reader(f3)
    for j in csve:
            if j[2] == r:
                print("Player with maximum score is:", j[1],
                      "who's score is:", j[2])
            elif j[2] == d:
                print("Player with minimum score is:", j[1],
                      "who's score is:", j[2])
    f3.close()
Seach()
```

STACK

Push: To add data to a stack
pop: To remove data from a stack
peek: To display the top element of the stack
display: To display all the stack values in order of first in

STACK-WORKING:

Q) To push, pop, peek, display items in a stack normally.
(items can be anything, we consider them as numbers in this program)

→
```
def push(item):
    stack.append(item)
    top = len(stack)-1

def pop():
    if stack == []:
        return "Underflow"
    else:
        item = stack.pop()
        if len(stack)==0:
            top = None
        else:
            top = len(stack)-1
        return item

def peek():
    if stack == []:
        return "Underflow"
    else:
        top = len(stack)-1
        return stack[top]

def Display():
    if stack == []:
        print("The stack is empty")
    else:
        top = len(stack)-1
        print("The elements of the stack are:")
        for i in range(top,-1,-1):
            print(stack[i])
```

```
stack=[]
top = None
while True:
    print ("1 for push || 2 for pop || 3 for peek || 4 for Display
           || 5 for Exiting ")
    ch= int(input("Enter the option you ~~desire~~ desire:"))
    if ch==1:
        item = input("Enter the value you wish to push:")
        push(item)
    elif ch==2:
        item= pop()
        if item == "Underflow":
            print("The stack is already empty")
        else:
            print("Deleted item:", item)
    elif ch==3:
        ~~if~~
        item=peek()
        if item == "Underflow":
            print("The stack is empty")
        else:
            print("The top element is:", ~~top~~ item)
    elif ch==4:
        Display()
    elif ch==5:
        break
    else:
        print("Invalid Choice")
```

STACK-WORKING:

Q) To recieve rollno, marks, names of students and store it in a dictionary, and to append only those dictionaries (info of studeents) to the stack where marks > 70.

→
```
def push(item):
    stack.append(item)
    top = len(stack)-1

def pop():
    if stack==[]:
        return "Underflow"
    else:
        item=stack.pop()
        if len(stack)==0:
            top=None
        else:
            top=len(stack)-1
        return item

def peek():
    if stack==[]:
        return "Underflow"
    else:
        top=len(stack)-1
        return stack[top]

def Display():
    if stack==[]:
        print("The stack is empty")
    else:
        top=len(stack)-1
        print("The elements of stack are:-")
        :
```

```
        for i in range (top,-1,-1):
            print (stack[i])
stack=[]
top = None
while True:
    print ("1. Push || 2. Pop || 3. Peek || 4. Display || 5. Exit")
    ch = int (input ("Enter your desired choice :"))
    if ch==1:
        item = {}
        rollno = int (input ("Enter roll no :"))
        marks = int (input ("Enter marks: "))
        name = int (input ("Enter name :"))
        item [rollno] = [name, marks]
        if item [rollno][1] > 70:
            push(item)
            item = {}
    elif ch==2:
        item=pop()
        if item == "Underflow":
            print ("The stack is empty")
        else:
            print ("The Deleted item is:", item)
    elif ch==3:
        item = peek()
        if item == "Underflow":
            print ("The stack is empty")
        else:
            print ("The top element is:", item)
    elif ch==4:
        Display()
    elif ch==5:
        break
    elif ch==6:
        print("Invalid choice")
```

```
if ch==1:
    d={}
    Rollno=
    marks=
    name=
    d[rollno]=[name,marks]
    if d[rollno][1]>70:
        item=(rollno,name)
        push(item)
        d={}
```

If you only want to push name and roll no. & not the whole dictionary.

OR,

```
if d[rollno][1]>70:
    item=(rollno, d[rollno][0])
    push(item)
    d={}
```

Q) Write a menu based program to add, peek, delete and display the records of LIBRARY using list as stack data structure in python. Record of LIBRARY contain the information BOOKID, BOOKTYPE, and BOOKCOST.

```
→ def push (records):
        LIBRARY
      ~~stack~~.append (records)
        LIBRARY
      top = len (~~stack~~) - 1

def pop():
    if ~~stac~~ LIBRARY == []:
        return "Underflow"
    else:
        records = LIBRARY.pop()
        if len(LIBRARY) == 0:
            top = None
        else:
            top = len (LIBARY) - 1
        return records
def peek():
    if LIBRARY == []:
        return "Underflow"
    else:
        top = len (LIBRARY) - 1
        return LIBRARY[top]                    #LIBRARY[top]

def DISPLAY():
    if LIBRARY == []:
        print ("The LIBRARY is empty")
    else:
        top = len (~~#~~ LIBRARY) - 1
        :
        :
```

```
        print ("The records in the LIBRARY are:")
        for i in range (top,-1,-1):
            print (LIBRARY[i])

LIBRARY=[]
top=None
while True:
    print ("1.Push || 2.Pop || 3.Peek || 4.Display || 5.exit")
    ch= int(input ("Enter your choice"))
    if ch==1:
        records=[]
        BOOKID = int(input ("Enter Book ID:"))
        BOOKTYPE = input ("Enter Book type:")
        BOOKCOST = float(input ("Enter Book Cost:"))
        records = [BOOKID, BOOKTYPE, BOOKCOST]
        push(records)
        records = []
    if ch==2:
        records = pop()
        if records == "Underflow":
            print ("The LIBRARY has no records")
        else:
            print ("The deleted record is:", records)
    if ch==3:
        records = peek()
        if records == "Underflow":
            print ("The LIBRARY has no records")
        else:
            print ("The top record is:", record)
    if ch==4:
        display()
    if ch==5:
        break.
```

Exception is for Like function, always ' ' even in format place (`%')

According to next page,

i) select records from emp table where salary is greater than a value given by the user, here a=int(input("Enter salary:"))

iii) select records from emp where gender is equal to the gender given by the user, here x = input("Enter Gender:")
also, since `x' is a string, in old style: `%s'
in new style : `{}'

Q) select those records from student where name starts with that letter which the user gives, and age matches what user gives.
A = input("Enter a letter:")
B = int(input("Enter Age:"))
dbcursor.execute("select * from student where name like '{}' and age = {}".format('A%', B)
or ..name like '%s' and age = %s"%('A%', B)

Q) In the second example, instead of doing 'so long' for the first query, we could have just:
y = int(input("Enter year:"))
gender = input("Enter gender:")
input(year, gender)
query = "select * from Employee where gender='{}' and DOJ like '{}'"

① dbcursor.execute(query, input)
or
② dbcursor.execute("select * from Employee where gender='{}' and DOJ like '{}'".format(g,y)
If we already know the gender 'F', then:
③ dbcursor.execute("select .... like '{}'.format('F','y')
or
④ dbcursor.execute("select * from Employee where gender='%s' and DOJ like '%s'"%(g,y)

Python-MySQL Interface: Basic Theory

1. The 'connect()' function of mysql.connector package creates a connection object which will act as the connector to the database specified.
2. The 'Database cursor instance' helps in executing the SQL Query, therefore facilitates the complete access to the resultset or row wise access as required.
3. The 'fetch' function helps extracting data from the cursor object.
4. The 'close()' function helps closing the connection on the connection object.

5. Connection object : A database connection object controls the connection of the database.
6. Cursor Instance : A control structure that runs SQL query using the execute() and contains the entire resultset returned by the query.
7. Resultset : A set of records that are fetched from the database by executing an SQL query.
8. fetch functions:
   i) <cursor>.fetchall() → returns all the records returned from the query, each record being a tuple.
   ii) <cursor>.fetchone() → returns one result from the resultset.
   iii) <cursor>.fetchmany(n) → returns the number of records specified by 'n'
      > Records are returned as individual tuples in a list.
      > if n is not specified, only one record is returned.
9. Rowcount: An attribute / property to display the number of records returned by the corresponding fetch function.

String template with %s :

- dbcursor.execute("select * from emp where salary > %s" % (a,))
- dbcursor.execute("select * from emp where salary > %s and age < %s" % (a,b))
- dbcursor.execute("select * from emp where gender = '%s'" % (x,))
- dbcursor.execute(query, input) where query = "select...." and input = value taken as tuple

String template with format()

- dbcursor.execute("select * from emp where salary > {} and age < {}".format(a,b))
- dbcursor.execute("select * from student where name like '{}'".format('A%'))

Commit()

- Insert and update queries can also be executed through python using execute().
However since these statements make changes in the table, commit() should be used along with the connection object to implement the changes.

Ex:

```
s = "update emp set salary = {} where age < {}".format(10000, 25)
db cursor.execute(s)
Obj.commit()
```

Hence commit() should be run along with the connection object for queries that change the data of the selected table so that the change is reflected on the table.

A Sample Program.

```
#mysql connector sample program

import mysql.connector as mc                    #Importing package
obj = mc.connect (host = "localhost", user="root", passwd="1234",
                  database = "myproj")          #Creating connection object
if not obj.is_connected():                      #checking whether connection is established
    print("Not connected successfully")
dbcursor = obj.cursor() # database cursor       # Creating a database cursor.
dbcursor.execute ("select * from student") #Executing SQL query
data = dbcursor.fetchall()                      #fetch function to retrieve the result set
count = dbcursor.rowcount #rowcount - an attribute to show how many rows extracted using fetch function
print ("Total records::", count)
print (data)                                    #displayed as a list with each record as a tuple
for i in data:
    print(i)                                    # each record as a tuple in a new line
obj.close()                                     # connection object is closed.
```

Q) A table "Event" in a database "Programme" has the following fields & datatypes:

E_No → char(4)
E_Name → varchar(15)
NOP → int
E_Date → Date

WAP to print the following →

(a) To fetch only those records from the "Event" table where NOP is more than 10.

(b) To display the table contents in ascending order of E_date.

→
```
import mysql.connector as mc
obj = mc.connect(Host="localhost", user="root", passwd="1234",
                 database="Programme")
if not obj.is_connected():
    print("Not connected successfully")
dbcursor = obj.cursor()
dbcursor.execute("select * from Event where NOP>10")
data1 = dbcursor.fetchall()
print(data1)
dbcursor.execute("select * from Event order by E_date asc")
data2 = dbcursor.fetchall()
print(data2)
obj.close()
```

9) A table "Employee" in a database "ABC Corps" has the following fields and datatypes:

Emp_NO → char(4)
Emp_Name → varchar(15)
Gender → char(4)
Salary → int
DOJ → date

Design a program to do the following:

(a) Accept an year from the user in the form "yyyy". Display all "female" employees who joined the organisation that year.

(b) To display gender wise maximum & minimum salary.

```
→ import mysql.connector as mc
obj = mc.connect(host = "localhost", passwd = "1234", user = "root",
                 database = "ABC Corps")
dbcursor = obj.cursor()
dbcursor.execute("select * from Employee where gender = "F" ")
data1 = dbcursor.fetchall()
year = int(input("Enter for the year you are looking for: "))
print("Female employees who joined in", year, "are:")
for i in data1:
    k = str(i[4])
    w = k[0:4]
    if str(w) == str(year):
        print(i[1])          # because Emp_Name is in i[1] position
print(" ")
print("Gender wise maximum & minimum salary:")
dbcursor.execute("select gender, max(salary), min(salary)
                  from Employee group by gender")
data2 = dbcursor.fetchall()
for g in data2:
    print(g)
obj.close()
```

Q) A table "customer" in the database "prog" has the following fields & types: serial number → int
name → varchar(25)
address → varchar(60)

a) WAP to print the address of those customer where the word "way" is contained in the address field.

b) to print the first 5 records from the table "customer".

—

```
import mysql.connector as mc
obj = mc.connect(host="localhost", passwd="1234", user="root",
                 database="prog")
if not obj.is_connected():
    print("Not connected successfully")
dbcursor = obj.cursor
dbcursor.execute("select address from customers where address
                 like '%way'")
data1 = dbcursor.fetchall()
print(data1)
dbcursor.execute("select * from customer")
data2 = dbcursor.fetchmany(5)
print(data2)
obj.close()
```

or

```
dbcursor.execute("select * from customer limit 5;")
data2 = dbcursor.fetchall()
print(data2)
```

—

~~Old style : %s.~~
New style : {} format.

~~i) dbcursor.execute("select~~
ii) To select those records from table "emp" where salary > ₹70 and age is less than 30yrs.

Sol → a = int(input("Enter salary:"))
b = int(input("Enter age"))
dbcursor.execute("select * from emp where salary > {} and age < {}".format(a,b))

or,

dbcursor.execute("select * from emp where salary > %s and age < %s" %(a,b)

So, in a way, : 1st {} → %s, 2nd {} → %s, .format(a,b) → %(a,b)

If the variable ~~(a or b)~~ ~~suppose~~ (a) is a string, we will put '%s' or '{}'

Another way is :

query = "select * from emp where salary > %s and age < %s"
a = int(input("Enter salary:"))
b = int(input("Enter age:"))
input (a,b)
dbcursor.execute(query, input)

Suppose, we already know 1 variable amongst the two, then :

~~ifooo~~ dbcursor.execute("select * from emp where ~~gender~~ salary > {} and age < {}".format(70, b))

— ex : first page.

STRING MANUPULATION AND METHODS.

1. len(string) → It returns the length of its argument string.
   ex: >>> r: "affsfvsfs"
   >>> len(r) → 9
   r = """sfvf_ffs"fs"ff_dsfsf"""
   >>> len(r) → 20

2. string.capitalize() → It returns a copy of the string with its first character capitalized.

3. string.count("sub", starting index, ending index)
   → it returns the number of occurances of the substring "sub" in the whole string. If to find the substring between 2 indexes, we can use starting & ending index.

4. string.find("sub", starting index, ending index) -
   → It returns the lowest index in the string where the substring sub is found within the slice range of start & end.
   Returns -1 if sub is not found.

---

1) len(string)
2) string.capitalize()
3) string.count("sub", si, ei)
4) string.find("sub", si, ei)
5) string.index("sub", si, ei)
6) string.isalnum()
7) string.isalpha()
8) string.isdigit()
9) string.islower()
10) string.isspace()
11) string.isupper()
12) string.lower()
13) string.upper()
14) string.lstrip()
    string.rstrip()
    string.strip()
15) string.startswith("sub")
    string.endswith("sub")
16) string.title()
17) string.istitle()
18) string.replace("old", "new")
19) string.join(string iterable)
20) string.split(string/char)
21) string.partition(<sep/string>) sep is seperator.

22) a += i (to append 'i' in string 'a')

LIST MANUPULATION :

1. len (list)
2. list ([sequence])
3. list.index(item)
4. list.append(item)
5. list.extend(list)
6. list.insert(pos, item)
7. list.pop(index)
8. list.remove(value)
9. list.clear()
10. list.count(item)
11. list.reverse()
12. list.sort()
13. sorted(iterable sequence, [reverse = False/True])
14. min(list)
    max(list)
    sum(list)

TUPLE:

1. len(tuple)
2. max(tuple)
3. min(tuple)
4. sum(tuple)
5. tuple name.index(item)
6. sequence name.count(object)
7. sorted(iterable sequence, [reverse = False/True])
8. tuple(sequence)

Dictionaries:

1. len(D)
2. D.get(key, [default])
3. D.items()
4. D.keys()
5. D.values()
6. dict.fromkeys(key sequence, [value])
7. dict.setdefault(key, value)
8. D.update(other dictionary)
9. dict.copy()
10. dict.pop(key, value)
11. dict.popitem()
12. D.clear()
13. sorted(dict, [reverse = False/True])
14. max(dict)
    min(dict)
    sum(dict)
15. del D[key]

Functions :
Introduction.

* The value entered in a function is called a parameter.

WAP to check whether the given number is even or odd.

→
```
def nEven(n):
    if n%.2==0:
        print(n," is even ")
    else:
        print(n, "is odd")
n=int(input(" Enter a number :"))
nEven(n)
```

* Top Level Statements : statements which are not any part of any user defined function, falls in common execution area. The segment with top level statements are named "_main_" by default by Python)

*

Call by Value function	Call by reference function
i> No change in arguments / actual parameters even if there is a change in formal parameters.	i> Actual parameters / arguments are modified if there is change in formal parameter
ii> Immutable types implement Call by value mechanism.	ii> Mutable types implement Call by reference mechanism.

Normal Info:

$a+=b \Rightarrow a = a+b$

Arithmetic Operators: +, -, *, /, %, **, //
relational operators: <, <=, >=, >, != or <>, ==
assignment operators: =, +=, -=, *=, /=, %=, **=, //=
Logical operators: not, and, or
bitwise operators: (&, |, ^, ~)
membership operators: in, not in
identity operators: is, is not

Operators: (quotient in decimal): 5/2 = 2.5
(quotient in whole no): 5//2 = 2
(remainder): 5%2 = 1

Statistics Module.

	Function	Purpose	Example
1.	statistics.mean(seq)	Returns the average value of the sequence of values passed	l1=[2,4,8,2,4,6,9,2,3,7] statistics.mean(l1) 4.7
2.	statistics.median(seq)	Returns the middle value of the sequence.	l1=[2,4,8,2,4,6,9,2,3,7] statistics.median(l1) 5
3.	statistics.mode(seq)	Returns the most repeated value of the sequence.	l1 = [1,1,2,3,4,5,6,7,8,9] statistics.mode(l1) 2

Random Module:

Function	Purpose	Example.
1. random.random()	Generates a random float number between 0.0 and 1.0 which will always be less than 1.0. The function doesn't need any argument.	>> import random >> random.random() 0.6451736848075733
2. random.randint(a,b)	Returns a random integer between the specified integers.	>> import random >> random.randint(1,100) 95 >> random.randint(1,100) 49
3. random.randrange (start)(stop, step)	Returns a randomly selected element from the range created by the start, stop and step arguments. The value of start is 0 by default. Similarly, the value of step is 1 by default.	>> import random. >> random.randrange (1,10) 2 >> random.randrange (1,10,2) 5 >> random.randrange (0,101,10) 80.

Q) What possible output(s) are expected? Also specify the max ~~min~~ value that can be assigned to each of the variables FROM and TO.

```
import random
AR = [20,30,40,50,60,70];
FROM = random.randint(1,3)
TO = random.randint(2,4)
for K in range (FROM, TO+1):
    print(AR[K], end = "#")
```

\# 1 2 3 max-3
\# 2 3 4 max-4
(1/2/3, 3, 4, 5) # will go 1 less
20, 2, 3, 4

(i) 10# 40# 70# (ii) 30#40#50/# (iii) 50#60#70#
(iv) 40#50#70#

# Fruitful function:	Void function.
1. The function which returns any value or gives result after execution.	1. The function which do not return any value.
2. While writing fruitful functions we accept a return value and so we must assign it to a variable to hold return value.	2. In void function, we can display a value on screen but cannot return a value. A void func. may or may not have a return statement, and if void function has return statement then it is written without any expression.

Scope of variable.

→ The part of the program from a variable is accessible or recognised is called the scope of a variable.

Two types of scopes of variables are:

1) Global Scope : ~~A var~~	Local Scope.
1) A variable / an object declared in the top level segment ("__main__") of the program are said to be in Global Scope.	1) A variable / an object declared of any user defined function is said to be in local scope.
2) These variables are recognised throughout the program including user defined functions	2) The usage / accessibility of that variable is only within the function where it is declared.

* Python follows the pattern "LEGB" while resolving the scope of a name (variable / function):

L - Local Scope

E - Enclosing Scope

G - Global Scope

B - Built in scope.

More on Scope:

If a user defined function has a variable whose name is same as another variable in global scope, then it always refers to the local variable unless mentioned otherwise. For eg:

```
def sum(x,y):
        s = x + y    # in this func, 's' is a local variable
        print(s)     # here the value of local 's' will be displayed
        #.....
n = input(.....)
s = input(....)  # This is the global 's'
sum(n,s)
print(s)    # Here the value of s will be shown which is entered by
              the user.
```

If a programmer wants to use the value of global 's' in such a func (function having a same named variable as of the global scope) then the global statement should be specified. For eg:

```
def sum(x,y):
    global s
    s = x+y
    print(s)  # Here the value of global s will be modified and the
                sum will be stored
    #....
n = input(....)
s = input(....)
sum(n,s)
print(s)  # Here the modified value of s will be stored and not the
            one entered by the user.
```

#	ARGUMENTS	PARAMETERS (variables)
	→ The values which are sent in a function call statement. For eg: even(n); where n in known as the argument.	→ The values which are there in the function header while defining a function. For eg: def even(n): # where n is known as the parameter.
	→ Also known as actual parameter or actual arguments.	→ Are known as formal parameters or formal arguments.

* Types of formal arguments:

- Positional arguments (Required arguments) -
  argument passed to a func in correct positional order.
- Default arguments:
  that assumes a default value if a value is not provided to the argument while calling the function.
- Keyword arguments (named argument):
  the caller identifies the arguments by the parameter name

---

i) for i in n: # n is a list, tuple, dictionary, file.

ii) for i in range(n): # will start from zero (0, n) to exactly 'n'. So exactly 'n' times the loop will go.

iii) for i in range (n, t): # will start from 'n' to 't-1' value.

4) for i in range (len(n)): # n is a list or string.
if len(n) = 4;
loop will execute for
0, 1, 2, 3 (😐 like 4 times)

Programs:

Q) WAP to display the perfect square numbers from a list of integers.

→
```
def Sq(H):
    for i in H:
        for j in range(2,i):
            if j*j == i:
                print(i)
H = eval(input("Enter a list of numbers:"))
Sq(H)
```

Q) WAP ~~to~~ using a dictionary to ~~count~~ convert the number entered by the user into its corresponding number in words. Like '876' to 'Eight Seven Six'.

```
a = input("Enter a number:")        #String taken, not int.
dict = {"0":"Zero", "1":"One", "2":"Two", "3":"Three",
        "4":"Four", "5":"Five", "6":"Six", "7":"Seven",
        "8":"Eight", "9":"Nine"}
for i in a:
    print(dict[a], end=" ")
```

Q) WAP to accept a ~~dictionary~~ sentence, a character to be replaced and a character to be replaced with, from the user. A function Replc(<str1>, <ch1>, <ch2>) should have the mentioned parameters and will replace every occurrence of <ch1> with <ch2> in <str1>.

for eg: str1 = "TECHNOLOGY"
ch1 = "O"
ch2 = "A"
Output: "TECHNALAGY"

→ ~~def~~ Replc (~~str1~~, ch1, ch2):
~~for~~ i ~~in~~ str1:
if

→ def Replc (str1, ch1, ch2):
A = " "
for i in str1:
if i == ch1:
i = ch2 # or just A+ = ch2
A+ = i
elif i != ch1:
A+ = i
print(A)
str1 = input ("Enter string:")
ch1 = input ("Enter value to be replaced:")
ch2 = input ("Enter value to replace with:")
replc (str1, ch1, ch2)

Q) WAP to accept a list of numbers and display only the magic numbers. using a user-defined func. MAGIC N(list1) ~~printing~~.

```
def MagicN (list1):
    for i in list1:
        if i%.9==1:
            print(i)
list1 = eval (input("Enter list:"))
MagicN(list1)
```

Q) To check whether a number is an armstrong number of not:

$153 = 1^3+5^3+3^3 = 153$

→
```
def armstrong(a):
    x=0
    z=0
    copy=a
    while a>0:
        x = a%.10
        y = x**3
        z = y+z
        a = a//10
    if z==copy:
        print (a,"is a armstrong number")
    else:
        print (a,"is not an armstrong number")
a=int(input("Enter number:"))
armstrong(a)
```

NUMBER TYPES:

(1) ARMSTRONG: 153 example.
The sum of cube of each digit is equal to that number

(2) PALINDROME

(3) FIBBONNACCI

(4) MAGIC: is divisible by 9

(5) Niven: The sum of the digits of the given number divides that number)
18 = 1+8 = 9 and 18 % 9 == 0.

(6) DUCK (7) PRONIC (8) PERFECT (9) NEON

NIVEN Numbers:

def niven

Niven: An integer that is divisible by the sum of its digits. (21 = 2+1 = 3
21 % 3 = 0)

(6) DUCK: Has 'zero's present in it.

(7) PRONIC / Rectangular: numbers which are product of 2 consecutive numbers.
n * (n+1)

(8) NEON: where the sum of the digits of square of the number is equal to the number.
$9^2 = 81$ ⇒ 8+1 = 9

Check weather a number is Niven or not.

27 = 7+2 = 9

27 % 9 = 0

$$9\overline{)27}\ 3 \quad 27 \quad 0 \qquad 10\overline{)27}\ 2 \quad -20 \quad 7 \qquad 10\overline{)2}\ 0 \quad -0 \quad 2$$

```
→ def Niven(a):
    x=0
    y=0
    z=0
    c=a
    while a>0:
        x = a%10
        y = a//10
        z = x+z
        a = y
    if c%z==0:
        print("The number:",c,"is niven")
    else:
        print(c,"is not niven")
```

	27>0	2>0
x = a%10	7	2
y = a//10	2	0
z = x+z	17	9
a = y	2	0

BASIC:

In Range.

```
for a in range (90, 9, -9):
    print(a)
```

In While loop:

```
a = 90
while a>9:
    print a
    a = a-9
```

```
for a in range (25, 500, 25):
    print (a)
```

```
a=25
while a<500:
    print (a)
    a = a + 25.
```

CLASS 11 BASICS

- Every object has an identity, type and value.
- Mutable types : list, dictionary, set
  Immutable : int, float, complex, boolean, string, tuple
- Python Character set include : letter, digits, special Symbols, white spaces, ASCII and Unicode
- Tokens : The smallest individual unit in a program is called a 'token'. These are :
  Keywords, Identifiers, Literals, Operators, Punctuators
- 33 Keywords
- Identifiers are names given to variables, objects, class, functions etc.
  → i> Non keyword with no spaces
  ii> Names can contain letter, numbers and underscore (_)
  iii> Names cannot start with a numbers.
- Literals : Boolean - True or False Special - None
- Escape Sequence : A sequence of characters that don't represent themselves when used inside string literal or character.
  \n → New line character
  \t → Horizontal tab
- Punctuators : Symbols used to organise statements { \, #, ", :, @, [ ]
- Variable Internals : type(), id()

---

1) for K in range (2,10,2):
   print(K) → 2 4 6 8

2) for K in range (2,10,-2):
   print(K) → Empty

3) for K in range(10,2,-2):
   print(K) → 10 8 6 4

4) for K in range (-2,-10,-2):
   print(K) → -2 -4 -6 -8

5) for K in range (-2,-10,2):
   print(K) → Empty

6) for K in range (-10,-2,2):
   print(K) → -10 -8 -6 -4

• LOGICAL OPERATORS :

'or' operator : Returns TRUE is one of the expressions is true
Returns FALSE if both expression are false.

'and' operator : Returns TRUE if both expressions are true.
Returns FALSE if eith of the expressions is false.

• Operator Associativity : BODMAS

• Error : Bug in the code that causes irregular output or stops the program from execution.
Exception : An irregular, unexpected situation occuring during execution on which the programmer has no control.

• Errors :

Compile Time Errors : Occurs during time of compilation of code
> Syntax error : rules of programming language is misused
Eg : if a=5 : # a==5.
> Semantics : Statement is not meaningful
Eg : a+b=c # c=a+b
> Type : wrong type of data given to a function.
Eg : a="Hi"
a**2

Runtime Errors : During the execution of program.
> opening a file which does not exist
> dividing a number by zero
> Logical errors : Undesired output produced

---

str = "Wah"
print (str*2) → WahWah

gg = (6, 7, 'm')
print (gg*2) → (6, 7, 'm', 6, 7, 'm')

ff = [2, 3, 4] → [2, 3, 4, 2, 3, 4, 2, 3, 4]
print (ff*3)

Syllabus:

1. Evolution of Networking: Introduction to computer networks, evolution of networking (ARPANET, NSFNET, INTERNET).
2. Data communication terminologies: Concept of communication, components of data communication ~~media~~ (sender, receiver, message, communication media, protocols), measuring capacity of communication media (bandwidth, data transfer rate), IP address, switching techniques (Circuit switching, Packet switching).
3. Transmission media: Wired communication media (Twisted Pair Cable, Co-axial Cable, Fibre-Optic cable), Wireless ~~tx~~ media (Radio, Micro, Infra
4. Network devices (modem, Ethernet Card, RJ45, Repeater, Hub, Switch, Router, Gateway, WIFI Card).
5. Network topologies and Network types: (PAN) LAN, MAN, WAN), (Bus, star, tree)
6. Network protocols: ——
7. Web services: WWW, HTML, XML, Domain Name, URL, website, web browser, web server, web hosting.

In HTML - Tag semantics and tag sets are fixed.
In XML - Tag semantics and tag set are not fixed.
i.e. Tags are defined.

Web hosting is a means of hosting web-server applications on a computer system through which electronic content on the internet is readily available to any web-browser client.

Communication And Networking Concepts

1. ARPANET -
2. NSF -
3. TCP/IP -
4. NIU -
5. NIC -
6. IP -
7. MAC Address -
8. TAP -
9. LAN -
10. MAN -
11. PAN -
12. CAN -
13. WAN -
14. VGM -
15. DGM -
16. UTP Cable -
17. STP Cable -
18. CAT-5 cable -
19. WWW
20. GHz -
21. Hz -
22. MHz -
23. KHz -
24. THz -
25. PDA -
26. LASER -
27. bps -
28. Bps -
29. Kbps -
30. Kbps -
31. mbps -
32. Mbps -
33. P2P -
34. MODEM -
35. AM -
36. FM -
37. PM -
38. A/F -
39. DTE -
40. DCE -
41. CTS -
42. CD -
43. RJ-45 -
44. AUI Connector -
45. BNC Connector -
46. SNA -
47. ISP -
48. FDDI -
49. ISDN -
50. VFIR -
51. NFS -
52. HTTP -
53. MIME -
54. SMTP -
55. WAIS -
56. FTP -
57. URL -
58. URI -
59. URN -
60. SLIP -
61. LCP -
62. NCP -
63. IPCP -
64. PPP -
65. GSM -
66. SMS -
67. SIM -
68. IDEN -
69. TDMA -
70. CDMA -
71. WLL -
72. 3G -
73. EDGE -
74. UMTS -
75. PSTN
76. E-mail
77. CC -
78. BCC -
79. BSNL -
80. VSNL -
81. HTML -
82. XML -
83. DHTML -
84. ASP -
85. JSP
86. VB Script -
87. PHP -
88. PERL -
89. OSS -
90. FLOSS -
91. GNU -
92. FSF -
93. W3C -
94. GPL -
95. OSI -
96. FOSS -
97. SSL -
98. UNCITRAL -
99. IPR -
100. FAT -
101. IMEI -

Some More Functions : Armstrong, palindrome, fibonacci, magic, niven, duck, pronic, neon, perfect,

Q) FACTORIAL OF A NUMBER:

→
```
def facto(n):
    z=1
    for i in range(1,n+1):
    z=z*i
    print("The factorial of",n,"is:",z)
n=int(input("Enter a number"))
facto(n)
```

Q) if number is prime or not:

→
```
def prime(n):
    for i in range(2,n):
        if n%i==0:
            print("Not a prime number")
        else:
            print("A prime number")
n=int(input("Enter number:"))
prime(n)
```

Q) Identify the armstrong number from a given list.

```
-> def armstrong(L):
       for i in L:
       x = 0
       for i in L:
           while c > 0:
           a = i % 10
           b = a ** 3
           x = x + b
           c = i //
```

```
10 | 153     10 | 15     10 | 1
   - 150        - 10        - 0
       3            5          1
```

```
-> def armstrong(L):
       for i in L:
           x = 0
           c = i
           while i > 0:
               a = i % 10
               b = a ** 3
               x = x + b
               i = i // 10
           if x == c:
               print("It is armstrong number")
```

```
-> def armstrong(L):
       x = 0
       c = copy
       for i in L:
           c = copy(i)
```

```
10 | 8208      10 | 820      10 | 82
     8200           820
        8             0
        ↓             ↓
       8^3           0^3
```

Q) Identify whether a given number is palindrome or not:

→
```
def palin(n):
    s=""
    c=n
    while n>0:
        a=n%10
        s+=a
        n=n//10
    if int(s)==c:
        print
```

```
12
10|121
 120
   1

1
10|12
 -10
   2

0
10|1
 -0
  1

12
10|125
 -120
    3
```

```
121>0      12>0      1>0
a=1        a=2       a=1
s=1        s=12      s=121
12         1         0
```

→
```
def palin(n):
    rev=0
    r=0
    copy=n
    while n>0:
        r=n%10
        n=n//10
        rev=(rev*10)+r
    if rev==copy:
        print("It is palindrome")
    else:
        print("Not palindrome")
n=int(input("Enter number"))
palin(n)
```

```
151>0
r=151%10=1
n=151/10=15
rev=(0*10)+1
=1

15>0
r=15/10=5
n=15/10=1
rev=(1*10)+5
=10+5
=15

1>0
r=1%10=1
n=1/10=0
rev=(15*10)+1
=150+1
151
```

COMPUTER SCIENCE

MAIN CLASS 12:

(PRACTICAL)

1. NORMAL PYTHON SUMS - CLASS 11 - (LIST, TUPLE, DICTIONARY)
2. PYTHON - FUNCTIONS (USER DEFINED FUNCTIONS)
3. PYTHON - FILE HANDLING (TEXT, BINARY, CSV)
4. PYTHON - STACK (ALSO USING LIST)
5. DBMS (SQL) - making Tables, Solving Queries, Output look
6. PYTHON - MYSQL CONNECTOR CONNECTIVITY (HOW TO CONNECT - PDF & PRAC FILE)
7. COMMUNICATION & NETWORKING CONCEPTS (Everything)
8. BASIC - RANDINT, RANDOM

THEORY: <<WITH EVERY CHAPTER, CHECK CBSE SYLLABUS SHEET>>

1. BASICS OF PYTHON: (ALL PYTHON PDFs of CLASS 11 AND OTHERS)
   : (ALL PDFs of Class 12)
2. FUNCTIONS THEORY: BUILT IN (LIST, TUPLE, DICTIONARY) (OF CLAS 11)
   USER-DEFINED IN MODULE
   USER-DEFINED IN PROGRAM
   DEF: (Arguments, Parameters, default parameters, positional parameter, function returning value, flow of execution. Keyword arguments, Call by value / Reference, scope of variable (global, local))
3. FILE HANDLING: PDFs of CLASS 12
   DEF of all 3 types, relative and absolute paths and their usage (school diary)

4. STACK - (push, pop, peak, display)
5. SQL - PRACTICAL FILE, PRAC QPS, PDF (See diary for all definations)
   Def: usage of different functions, differences,
   (alter, update, modify): all in the PDF
   cardinality, degree,
   (no. of rows) (no. of colourns)

   Def: types of Colourns: Primary, (equijoin & natural join)
6. CONNECTIVITY - (PDF, definations, how to use functions)
   (commit, roll back, savepoint)
7. COMMUNICATION & NETWORKING CONCEPTS - EVERYTHING

REFERENCES:

PRACTICAL FILE OF 12	ALL PDFs of 12
12 & 11 { ANSWER SHEETS OF PRAC	ALL PDFs of 11
12 & 11 { QUESTION PAPER OF PRAC	syllabus sheet of 12
NISHANTS CHATS	DIARY.

PROBS (HARD PRACTICE)

- meaning of null, not null, is not null, is null
- HAVING CLAUSE WITH WHILE
- STRING TEMPLATES WITH %s and % in connectivity.
- diff. between MODULE, LIBRARY AND PACKAGE
- with open (file, mode, buffering, encoding = NONE,
  errors = None, newline = None, closefd = True,
  opener = None)
- ALIASING.
